Détection d'une figure dans une image — Mini-projet, Option B : plusieurs classes de formes

Réseaux de neurones — TP et mini-projet

[Nom Prénom]

1. Objectif

Le TP consistait à entraîner un détecteur qui, sur une image RGB 64×64 contenant **une seule figure toujours identique** (un disque rouge), prédit la présence d'un objet et sa boîte englobante (c_x, c_y, w, h). L'**Option B** étend ce détecteur : le générateur produit des **cercles, carrés et triangles de couleurs variées**, et le réseau doit prédire la **classe** de la figure en plus de sa boîte. Le code se compose de :

fichier	rôle
`src/dataset_generator_2.py`	générateur : une figure par image, forme et couleur aléatoires
`src/detector_multiclass.py`	codage/décodage, dataset, modèle `MultiClassDetector`, perte, prédiction
`src/train_multiclass.py`	entraînement d'une époque, évaluation, courbes, script complet
`notebooks/mini_projet_option_B.ipynb`	démarche complète commentée : TP, extension, résultats, figures

Seules les bibliothèques du TP sont utilisées (`torch`, `numpy`, `PIL`, `matplotlib`).

2. Rappel : le détecteur du TP

Image $3 \times 64 \times 64$ → *backbone* de 3 blocs « conv 3×3 stride 2 + BatchNorm + ReLU » ($3 \to 8 \to 16 \to 32$ canaux, $64 \to 32 \to 16 \to 8$ pixels) → feature maps $32 \times 8 \times 8$ → tête conv 1×1 → $5 \times 8 \times 8$: pour chaque cellule $(t_o, t_x, t_y, t_w, t_h)$ → décodage → NMS → une boîte. Le problème est simplifié par rapport au cours : une seule figure, une seule classe, pas d'ancres, une seule échelle. Entraîné 30 époques sur 2 000 images (validation : 400 images), il atteint un **IoU moyen de 0,895** et **100 % d'IoU > 0,5** sur les cercles rouges.

3. Choix de conception

Générateur. La forme est tirée uniformément parmi `circle` (0), `square` (1), `triangle` (2) et la couleur parmi 5 couleurs (rouge, bleu, vert, jaune, violet). La couleur étant indépendante de la classe, le réseau ne peut pas « tricher » : il doit apprendre la **forme**. Le côté varie de 10 à 21 pixels, la position est aléatoire, le fond est un gris clair aléatoire. L'étiquette devient `"cx cy w h class_id"` (pixels).

Tête. La conv 1×1 passe de 5 à $5 + C = 8$ canaux par cellule : objectness t_o, boîte (t_x, t_y, t_w, t_h), et $C = 3$ logits de classe (z_1, z_2, z_3). La sortie reste un tenseur $8 \times G \times G$: une seule boîte par cellule, mais avec une classe.

Backbone. On conserve les contraintes du TP (sortie $32 \times 8 \times 8$, BatchNorm et non-linéarité à chaque couche) mais chaque étage reçoit une **seconde convolution** 3×3 **de stride 1** : $3 \to 16 \to 16$, $16 \to 32 \to 32$, $32 \to 32 \to 32$ (6 couches, 35 736 paramètres contre 6 309). Le champ réceptif et la capacité augmentent sans changer la taille de sortie. Le paramètre `deep=False` redonne exactement le backbone du TP, ce qui permet la comparaison de la section 6.

Prédiction. Le score d'une cellule est $\sigma(t_o)$; on applique la NMS (seuil 0,5) puis on garde la meilleure boîte. Sa classe est $\text{argmax}_k p(k)$ avec la probabilité $p(k)$ affichée sur les exemples.

4. Codage, décodage et perte modifiés

Codage (étiquette → cible). L'image de 64 pixels est découpée en une grille de 8 × 8 cellules de 8 pixels. La cellule *responsable* est celle qui contient le centre de la figure : colonne = partie entière de cx / 8, ligne = partie entière de cy / 8. Dans cette cellule, la cible contient **6 valeurs**, une de plus que dans le TP ; toutes les autres cellules valent 0.

valeur	signification	calcul
objet	une figure est présente	1
dx, dy	position du centre dans la cellule (entre 0 et 1)	partie décimale de cx / 8 et de cy / 8
w, h	taille de la boîte par rapport à l'image	w / 64 et h / 64
classe	**nouveau** : numéro de la forme	0 (cercle), 1 (carré) ou 2 (triangle)

Exemple : l'étiquette `25 14 16 16 1` (carré de 16 pixels centré en (25, 14)) donne la cellule ligne 1, colonne 3, avec la cible (1 ; 0,125 ; 0,75 ; 0,25 ; 0,25 ; 1).

Décodage (sortie du réseau → boîte et classe). Pour la boîte, rien ne change : une sigmoïde ramène les 5 premières valeurs entre 0 et 1, puis on revient aux pixels :

> centre x = (colonne + dx) × 8, centre y = (ligne + dy) × 8 ;
> largeur = w × 64, hauteur = h × 64.

Nouveau : les 3 dernières valeurs passent par un *softmax*, qui les transforme en 3 probabilités (cercle, carré, triangle) dont la somme vaut 1. La classe prédite est la plus probable.

Perte. On garde les deux termes du TP et on en ajoute un troisième pour la classe :

> **perte = BCE (objet) + 10 × MSE (boîte) + 1 × CE (classe)**

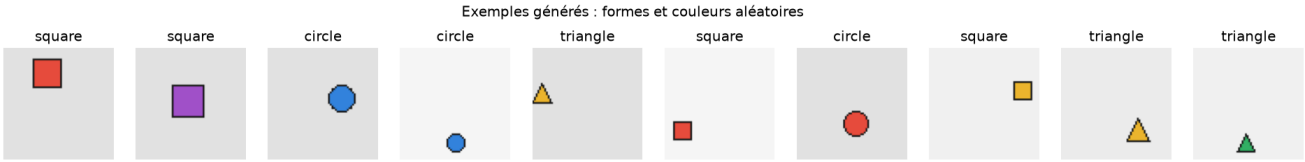
- **BCE** (entropie croisée binaire) : « y a-t-il un objet ? », sur les 64 cellules. La cellule positive reçoit un poids de 10 pour compenser le déséquilibre (1 cellule positive pour 63 négatives).
- **MSE** (erreur quadratique) : écart entre (dx, dy, w, h) prédits et vrais, sur la cellule positive seulement. Ces erreurs étant petites, on les multiplie par 10.
- **CE** (entropie croisée, **nouveau**) : vaut –log(probabilité donnée à la bonne forme) ; elle est presque nulle si le réseau est sûr et a raison, et grande s'il se trompe avec assurance. Elle n'est calculée que sur la cellule positive : ailleurs il n'y a pas d'objet, donc pas de classe à apprendre.

On n'optimise pas l'IoU directement : elle vaut 0 (donc aucun gradient) dès que les boîtes ne se chevauchent pas.

5. Protocole expérimental

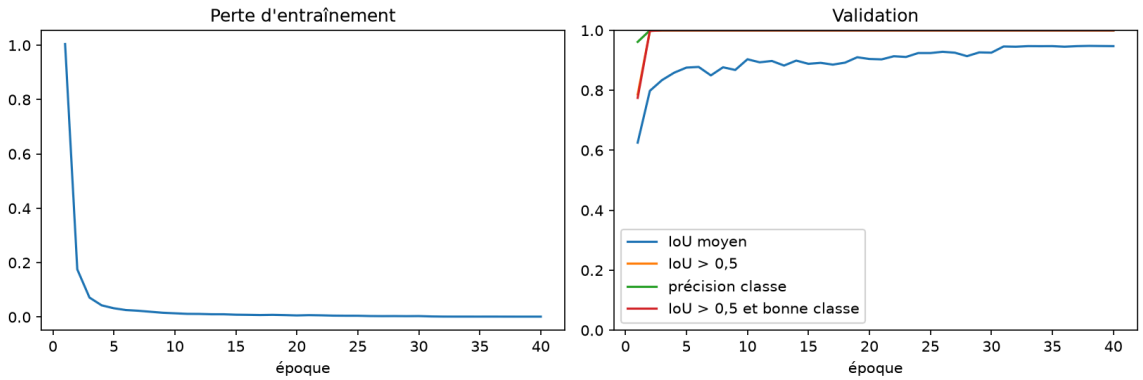
3 000 images d'entraînement et 600 de validation, générées avec des graines différentes. Adam, pas $3 \cdot 10^{-3}$ divisé par 10 à l'époque 30, batches de 64, 40 époques, graine fixe (reproductible). Métriques sur la validation : **IoU moyen**, **proportion d'IoU > 0,5**, **précision de classification**, **matrice de confusion** et proportion de **détections complètes** (IoU > 0,5 *et* bonne classe).

6. Résultats



Exemples d'images générées : forme et couleur aléatoires.

Courbes d'apprentissage. La perte décroît rapidement ; la classification est apprise en quelques époques, la localisation s'améliore plus lentement et gagne encore après la baisse du pas d'apprentissage (époque 30).

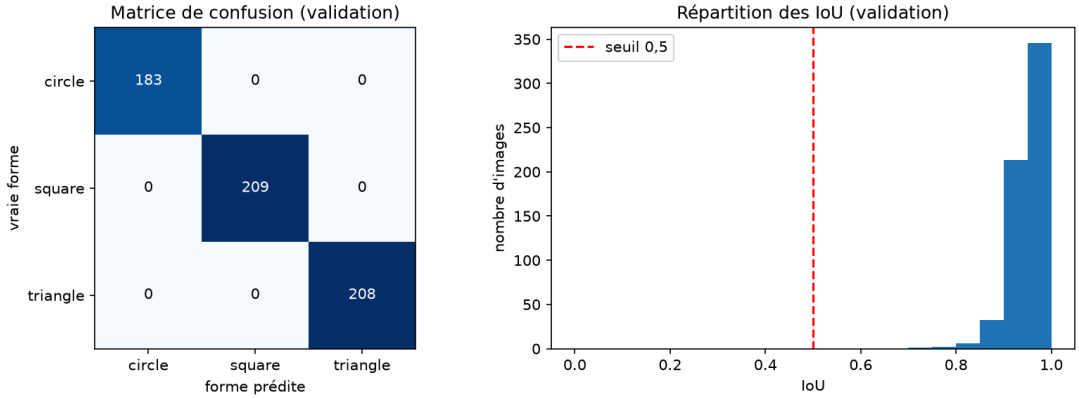


Perte d'entraînement et métriques de validation du modèle multi-classes (backbone profond).

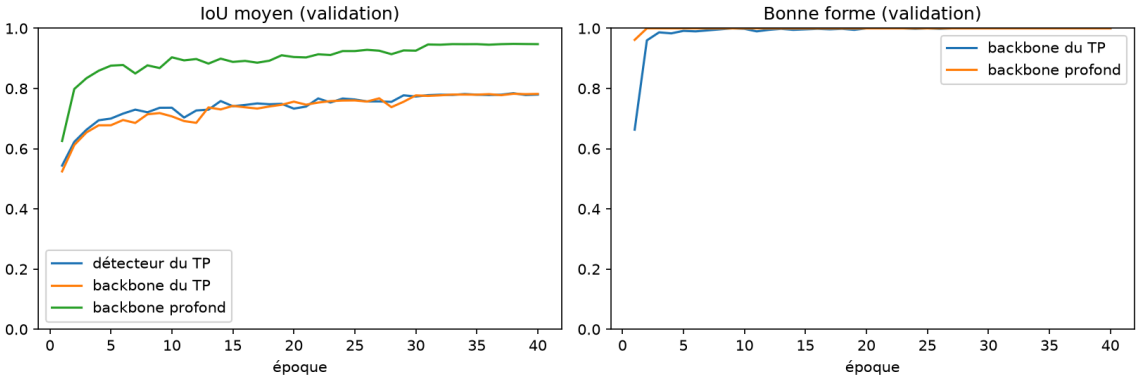
Résultats chiffrés (validation, 600 images).

modèle	IoU moyen	IoU > 0,5	classe correcte	paramètres
TP entraîné sur cercles rouges, sans réentraînement	0,366	26,2 %	—	6 309
TP réentraîné sur les données multi-classes (boîte seule)	0,784	96,3 %	—	6 309
Multi-classes, backbone du TP (`deep=False`)	0,777	95,5 %	99,8 %	6 408
Multi-classes, backbone profond	**0,949**	**100 %**	**100 %**	35 736

Pour le modèle final, la détection complète (IoU > 0,5 et bonne classe) est de **100 %** ; l'IoU moyen par classe vaut 0,953 (cercle), 0,951 (carré) et 0,944 (triangle), et la matrice de confusion est diagonale (183 cercles, 209 carrés, 208 triangles, aucune confusion). Selon la taille de la figure, l'IoU moyen passe de **0,933** (côté 10–13 px) à 0,954 (14–17 px) et **0,960** (18–21 px).

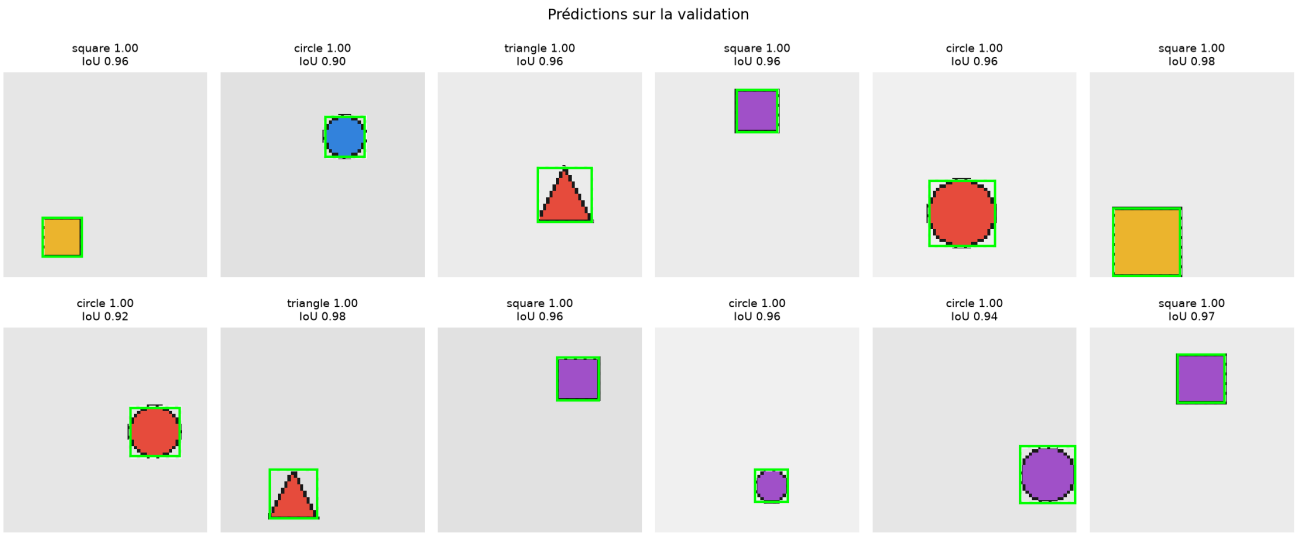


Matrice de confusion et distribution des IoU sur la validation.

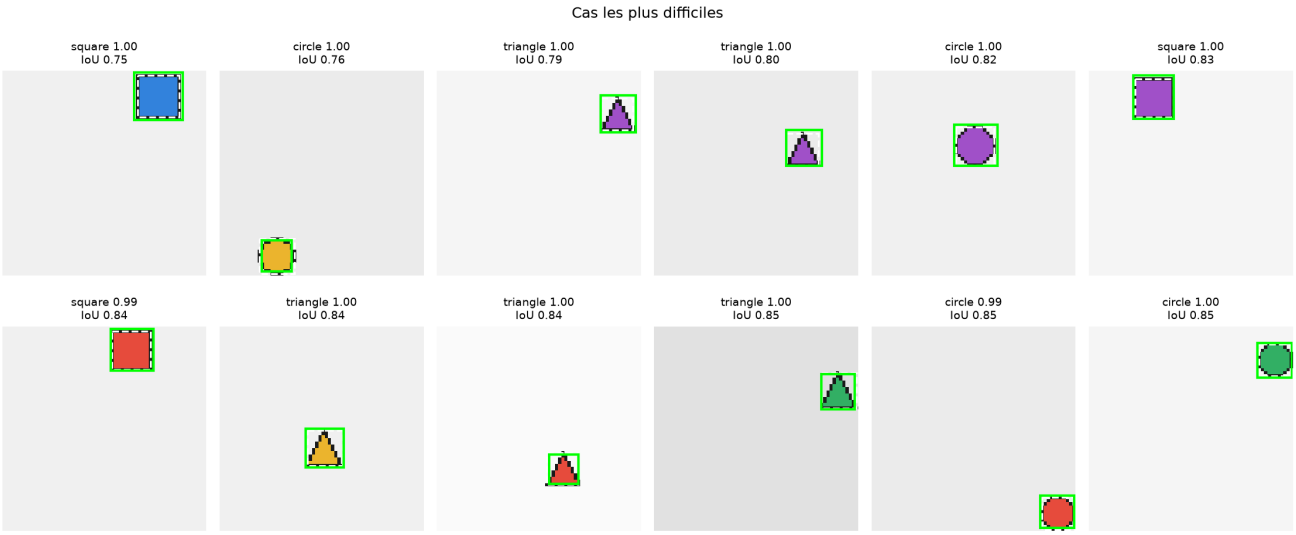


Comparaison des modèles : IoU moyen et précision de classification au fil des époques.

Exemples visuels. Boîte prédite verte (classe correcte) ou rouge (classe fausse), vraie boîte en pointillés ; titre : classe prédite, probabilité, IoU. Les cas « les plus difficiles » (plus faibles IoU) restent bien localisés (IoU ≥ 0,80) : l'erreur résiduelle est un décalage d'un ou deux pixels, surtout sur les **petites figures** et sur les figures **collées au bord de l'image**.



Prédictions sur des images de validation.



Les 12 cas de validation les plus difficiles (plus faibles IoU).

Analyse.

- **Le détecteur du TP ne généralise pas** : entraîné sur des cercles rouges, il n'obtient que 0,366 d'IoU sur les nouvelles formes et couleurs. Il faut réentraîner sur des données représentatives.
- **Ajouter la classe ne dégrade pas la localisation** : avec le même backbone, le modèle multi-classes (0,777) localise aussi bien que le détecteur du TP réentraîné (0,784), tout en classant 99,8 % des figures.
- **La classification est facile ici** : même le petit backbone du TP sépare presque parfaitement les trois formes. Le fond uniforme et les contours noirs rendent la forme très visible.
- **La profondeur du backbone améliore surtout la localisation** : l'IoU moyen passe de 0,777 à 0,949. Ce gain s'accompagne de 5,6 fois plus de paramètres ; une partie vient donc simplement de la capacité.

7. Limites et perspectives

- **Données très simplifiées** : une figure par image, toujours de largeur égale à la hauteur ($w = h$), sans rotation ni bruit, sur fond uniforme. Les scores proches de 100 % ne seraient pas atteints sur des images réelles, et la précision de classification ne permet pas de départager les modèles.
- **La NMS ne sert presque pas** : avec un seul objet, on garde la boîte de meilleur score. Avec plusieurs objets (Option C), il faudrait une NMS par classe et un seuil sur le score.
- **Une seule boîte par cellule** : deux figures dont les centres tombent dans la même cellule ne pourraient pas être détectées ; les ancres (Option A) répondraient à ce problème.
- **Score de détection** : on utilise $\sigma(t_o)$ seul ; un score $\sigma(t_o) \times \max_k p(k)$ tiendrait compte de l'incertitude de classe, utile pour calculer une mAP.
- **Pistes** : rendre les données plus difficiles (bruit, rotations, rapports w/h variés, formes proches comme losange/carré), augmentation de données, réglage de λ_{cls}, et calcul d'une mAP par classe.